More practice - Sorting and divide and conquer

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3	1
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Sorting and divide and conquer | 4 questions | tap a title to jump to it

M	1. Progressive Array Partitioning	1Point3Acres	2026-08-11
M	2. Check if all substrings are anagrams of words	PracHub	2026-03-11
M	3. Streaming Points: Emit K Within Distance	1Point3Acres	2026-06-23
H	4. Find safe travel intervals between planet influences	PracHub	2025-11-22

STUDY PATH: question 153 of 167 (more practice)

Priority score 3.5 | Sorting and divide and conquer | asked 70 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 1 of 4

Progressive Array Partitioning

ASKED 2026-08-11 | 70 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **NO LEETCODE FREQUENCY** | **PRIORITY 198/228**

ABOUT THIS QUESTION

Topic: More practice - Sorting and divide and conquer

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-11 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/array-partition-unique-frequencies-straights

QUESTION

EXAMPLES

Stage 1

```
[1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true because the
    four value-groups have different sizes.
[1, 2, 3] -> false because every value-group has size
    one.
```

Stage 2

```
[1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true.
[4, 5, 6, 7, 7, 8, 9] -> false.
```

APPROACH

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the groups are and the rule for each stage. From the examples, the first stage looks like: true when the counts of all distinct values are different from each other (1:1, 2:2, 3:3, 4:4 is true; all counts 1 is false). The second stage is not clear from the examples alone, so ask for its exact rule and test it on both examples before coding.

BRUTE FORCE TO SAY OUT LOUD

Try every way to group the elements and test the rule. Exponential.

OPTIMAL IDEA

For stage one: count each value with a hash map, then check that the multiset of counts has no repeats (a set of counts has the same size as the number of values). O(n). Keep the data model (a counter) easy to adapt, because the second stage arrives only after the first is written.

DRY-RUN THESE CASES

The four examples in the statement, an empty array, a single value, many values with equal counts.

LIKELY FOLLOW-UPS

Stage two, which changes the grouping rule; keep helper functions small so it is a local change.

PITFALLS

Coding stage two from a guess. Not reconciling the rule with both examples.

HOW TO THINK ABOUT IT

The statement arrives in stages, so keep the data model easy to adapt: count the values first, because any rule about 'groups of equal values' is a rule about the counts. Stage 1 is then 'are all counts different', which is a set-size comparison. Before stage 2, reconcile the new rule with every example; here a natural guess (LeetCode 659) contradicts the second example, which is exactly the situation where you ask.

Basis: Written. Stage 1 is inferred from the examples: true when every distinct value occurs a different number of times. The Stage 2 rule is NOT recoverable from the two examples, so only a candidate reading is shown; ask the interviewer for the exact rule and test it against both examples before coding.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    counts = Counter(a)
    for x, y in combinations(counts, 2):
        if counts[x] == counts[y]:
            return False
    return True
```

TIME	SPACE
O(d^2) where d is the number of distinct values	**O(d)**

WHY IT IS SLOW

O(d^2) pair checks for d distinct values. Comparing the number of distinct counts with the number of distinct values (a set) is O(n).

OPTIMAL SOLUTION · Counter, then compare the set of counts with the number of values

TIME	SPACE
O(n)	**O(d)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Stage 2 arrives. What do you do?

Do not guess. Write down what the second example pair implies, and check any candidate rule against BOTH. The common reading 'split into consecutive runs of length at least 3' gives true for [4, 5, 6, 7, 7, 8, 9], but the statement says false, so there is an extra condition (such as run lengths or a count rule) to ask about.

FOLLOW-UP 2 Keep the code adaptable between stages.

Put the counting in one helper and each stage's rule in its own small function that takes the counts. Stage 2 then replaces a function instead of rewriting the pipeline.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Progressive Array Partitioning
#
# Examples: Stage 1 [1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true
# because the four value-groups have different sizes. [1,
# 2, 3] -> false because every value-group has size one.
# Stage 2 [1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true. [4, 5, 6
,
# 7, 7, 8, 9] -> false.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Stage 2 arrives. What do you do?
# 2. Keep the code adaptable between stages.
#
# INTERVIEW NOTES
# The statement arrives in stages, so keep the data model
# easy to adapt: count the values first, because any rule
# about 'groups of equal values' is a rule about the
# counts. Stage 1 is then 'are all counts different',
# which is a set-size comparison. Before stage 2,
# reconcile the new rule with every example; here a
# natural guess (LeetCode 659) contradicts the second
# example, which is exactly the situation where you ask.
#
# BRUTE FORCE (say it first): Compare every pair of counts
# Count each value, then check every pair of counts for
# equality. Fine for small d and a clear statement of the
# rule. Time O(d^2) where d is the number of distinct
# values. Space O(d).


from collections import Counter


# BRUTE FORCE (say it first): Compare every pair of counts
# | time O(d^2) where d is the number of distinct values
def distinct_counts_brute(nums):
    counts = list(Counter(nums).values())
    return all(
        counts[i] != counts[j]
        for i in range(len(counts))
        for j in range(i + 1, len(counts))
    )


# BASE QUESTION (optimal): Counter, then compare the set
# of counts with the number of values
```

```
# Idea: All counts are different exactly when the set of
# counts has as many elements as there are distinct
# values. One pass to count, one pass over the counts.
def stage1(counts):
    # true when every value occurs a different number of
    # times: the counts have no repeats
    return len(set(counts.values())) == len(counts)


# FOLLOW-UP 1: Stage 2 arrives. What do you do?
# Answer: Do not guess. Write down what the second example
# pair implies, and check any candidate rule against BOTH.
def stage2_candidate(nums, min_len=3):
    # ONE reading of stage 2 (LeetCode 659): split the
    # array into runs of consecutive values, each at
    # least min_len long. Check it against BOTH examples.
    left, ends = Counter(nums), Counter()
    for x in nums:
        if left[x] == 0:
            continue
        left[x] -= 1
        if ends[x - 1] > 0:
            # extend a run that ended at x - 1
            ends[x - 1] -= 1
            ends[x] += 1
        elif left[x + 1] > 0 and left[x + 2] > 0:
            # start a new run x, x + 1, x + 2
            left[x + 1] -= 1
            left[x + 2] -= 1
            ends[x + 2] += 1
        else:
            return False
    return True


# HELPERS for the checks below
def counts_of(nums):
    # every rule about groups of equal values is a
    # rule about these counts, so compute them once
    return Counter(nums)


# each stage is one small function, so a new rule
# replaces one entry instead of the whole pipeline
RULES = {1: lambda nums: stage1(counts_of(nums)),
```

```
         2: stage2_candidate}


def check(stage, nums):
    return RULES[stage](nums)


if __name__ == "__main__":
    # stage 1 examples
    assert check(1, [1, 2, 3, 4, 2, 3, 3, 4, 4, 4])
    assert not check(1, [1, 2, 3])
    # stage 2 example that the candidate gets right
    assert check(2, [1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
    # the second stage-2 example says False, but this
    # reading says True: the rule is incomplete, so ask
    assert check(2, [4, 5, 6, 7, 7, 8, 9]) is True
```

NOTES FROM CANDIDATES

The interviewer presents Stage 2 only after the first implementation, so keep the initial data model easy to adapt.
Before coding the straight variant, reconcile the length rule with both examples; otherwise a superficially valid decomposition can contradict the expected result.
Evaluation emphasized the efficiency of the chosen data structures, not only whether the sample cases passed.
Solution skeleton
Stage 1: count each distinct value, then verify that the frequency values are themselves unique. A hash map plus a set gives O(n) time and O(u) space for u distinct values.
Stage 2: first pin down a group size k. For the fixed-size interpretation, reject when n % k != 0, process keys in ascending order, and whenever the smallest remaining key x has multiplicity c, remove c copies from every key in [x, x + k). Any missing count makes the partition impossible. Sorting the u keys gives O(n + u log u) time and O(u) space.
Do not let length-one groups make every input trivially valid unless the interviewer explicitly allows them; the negative example requires a stronger group-size contract.

PREPARATION

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

STUDY PATH: question 154 of 167 (more practice)

Priority score 3.31 | Sorting and divide and conquer | asked 223 days ago

MEDIUM | PracHub | Technical Screen | 2 of 4

★ TOP 75: rank 22 in the Medium and Hard priority list

★ Check if all substrings are anagrams of words

POSTED 2026-03-11 | 223 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 51.4 | PRIORITY 16/228

ABOUT THIS QUESTION

Topic: More practice - Sorting and divide and conquer

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #49 Group Anagrams

LeetCode match: No direct match; related LC 49 Group Anagrams

Posted on PracHub: 2026-03-11 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/check-if-all-substrings-are-anagrams-of-words

QUESTION

You are given a string **s** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **s** with length **>= 3** can be **rearranged (anagrammed)** into a valid dictionary word. Formally, for every substring **`t = s[i..j]`** with **`|t| >= 3`**, there exists some word **`w in dict`** such that **w** is a permutation of **t** (same multiset of characters). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**. - "Can be rearranged into a word" means the substring and the dictionary word have exactly the same character counts. Example:
- If **`t = "act"`**, it passes if the dictionary contains any of **`{ "act", "cat", "tac" }`**.

APPROACH

Precompute sorted-letter (or count-vector) signatures of dictionary words into a set; for each substring of length >= 3 compute its signature and look it up.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether every substring of length at least 3 must be rearrangeable into some dictionary word, the alphabet (letters only), and the dictionary size and word lengths.

BRUTE FORCE TO SAY OUT LOUD

For every substring, compare against every dictionary word by sorted letters. Very slow.

OPTIMAL IDEA

Preprocess the dictionary into a set of signatures, where a signature is the sorted letters or a 26-count tuple, and also record the set of word lengths present. For each start i extend j one letter at a time, updating a count vector incrementally; once the length is at least 3, look up its signature (skip lengths that no word has) and return false at the first miss. O(n^2 * 26) time.

DRY-RUN THESE CASES

A string shorter than 3, a failing length that has no dictionary words, a long repeated letter, all substrings valid.

LIKELY FOLLOW-UPS

Return the failing substring, or a larger alphabet (use a hash of counts).

PITFALLS

Sorting each substring from scratch, which costs an extra log factor. Forgetting that a signature must match the length too (a count tuple already does).

HOW TO THINK ABOUT IT

Anagrams share a signature: the multiset of letters (a sorted string or a 26-count tuple). So turn the dictionary into a set of signatures once, and test each substring by its signature. Extending a substring by one letter changes its count vector in O(1), so you do not rebuild the signature from scratch.

Basis: Written; every substring of length >= 3 must be a permutation of some dictionary word (compare Group Anagrams, LeetCode 49).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, words):
    for i in range(len(s)):
        for j in range(i + 3, len(s) + 1):
            key = sorted(s[i:j])
            match = [w for w in words
                     if key == sorted(w)]
            if not match:
                return False
    return True
```

TIME	SPACE
O(n^2 * W * L log L)	**O(1)** extra

WHY IT IS SLOW

Each substring is compared with every word and sorted again: O(n^2 * W * L log L). A set of word signatures, with the substring's count vector updated incrementally, makes each check O(alphabet).

OPTIMAL SOLUTION · Set of signatures, incremental count vectors

TIME	SPACE
O(n^2 * A) for alphabet size A	O(dictionary size)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A bigger alphabet or Unicode.

Replace the fixed array by a Counter and use a frozen tuple of sorted (letter, count) pairs, or a hash of the counts, as the signature. **Cost:** O(n^2 * distinct letters in a substring) with a Counter signature instead of a fixed array.

FOLLOW-UP 2 Only substrings of lengths that exist in the dictionary need checking.

Record the set of word lengths; a substring whose length is not in that set can never match, so you can return False the first time such a length is reached (assuming all substrings of length >= 3 up to n must match).

FOLLOW-UP 3 Subsets of the letters (any sub-multiset of a word) instead of a full permutation.

Then the test is containment of count vectors, which is a subset-of-multiset query; use a sorted signature trie or precompute all sub-multisets for short words.

FOLLOW-UP 4 Report the first failing substring instead of True or False.

Test substrings by length (shortest first) so the report is a smallest counterexample; the signature set makes each test one lookup after sorting the substring's letters.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check if all substrings are anagrams of
# words
#
# You are given a string s (letters only) and access to an
# English dictionary dict (a set of valid words). Return
# true if **every contiguous substring** of s with length
# **>= 3** can be **rearranged (anagrammed)** into a valid
# dictionary word. Formally, for every substring t =
# s[i..j] with |t| >= 3, there exists some word w in dict
# such that w is a permutation of t (same multiset of
# characters). Otherwise return false. Clarifications: -
# Substring means **contiguous**. - “Can be rearranged
# into a word” means the substring and the dictionary word
# have exactly the same character counts. Example: - If t
# = "act", it passes if the dictionary contains any of {
# "act", "cat", "tac" }.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A bigger alphabet or Unicode.
# 2. Only substrings of lengths that exist in the
# dictionary need checking.
# 3. Subsets of the letters (any sub-multiset of a word)
# instead of a full permutation.
# 4. Report the first failing substring instead of True or
# False.
#
# INTERVIEW NOTES
# Anagrams share a signature: the multiset of letters (a
# sorted string or a 26-count tuple). So turn the
# dictionary into a set of signatures once, and test each
# substring by its signature. Extending a substring by one
# letter changes its count vector in O(1), so you do not
# rebuild the signature from scratch.
#
# BRUTE FORCE (say it first): Compare each substring with
# every word's sorted letters
# For each substring, sort its letters and compare with
# the sorted letters of every dictionary word. Correct,
# and the dictionary is rescanned for each substring. Time
# O(n^2 * W * L log L). Space O(1) extra.


from collections import Counter
```

```
# BRUTE FORCE (say it first): Compare each substring with
# every word's sorted letters | time O(n^2 * W * L log L)
def _sig(w):
    return tuple(sorted(w))


def all_anagram_brute(s, words):
    sigs = [_sig(w) for w in words]
    return all(
        _sig(s[i:j]) in sigs
        for i in range(len(s))
        for j in range(i + 3, len(s) + 1)
    )


# HELPERS for the base question
ALPHABET = "abcdefghijklmnopqrstuvwxyz"


# HELPERS for follow-up 4
def signature(text):
    # the letters in sorted order: equal for anagrams
    return "".join(sorted(text))


# BASE QUESTION (optimal): Set of signatures, incremental
# count vectors
# Idea: Precompute the set of count-vector signatures of
# the dictionary words. For each start extend one letter
# at a time, update the count vector in O(1), and look up
# its tuple (once the length is at least 3).
def all_anagram_words(s, words, min_len=3):
    # every substring needs a dictionary word with the
    # same
    # letters. Count vectors are updated as the window
    # grows.
    index = {c: i for i, c in enumerate(ALPHABET)}
    sigs = set()
    for w in words:
        v = [0] * len(ALPHABET)
        for ch in w:
            v[index[ch]] += 1
        sigs.add(tuple(v))
    for i in range(len(s)):
```

```
        v = [0] * len(ALPHABET)
```

```
        for j in range(i, len(s)):
            v[index[s[j]]] += 1
            long_enough = j - i + 1 >= min_len
            if long_enough and tuple(v) not in sigs:
                return False
    return True


# FOLLOW-UP 1: A bigger alphabet or Unicode.
# Answer: Replace the fixed array by a Counter and use a
# frozen tuple of sorted (letter, count) pairs, or a hash
# of the counts, as the signature.
def all_anagram_words_any_alphabet(s, words, min_len=3):
    # Unicode or a big alphabet: a frozen Counter as
    # signature
    sigs = {frozenset(Counter(w).items()) for w in words}
    for i in range(len(s)):
        count = Counter()
        for j in range(i, len(s)):
            count[s[j]] += 1
            key = frozenset(count.items())
            if j - i + 1 >= min_len and key not in sigs:
                return False
    return True


# FOLLOW-UP 4: Report the first failing substring instead
# of True or False.
# Answer: Test substrings by length (shortest first) so
# the report is a smallest counterexample; the signature
# set makes each test one lookup after sorting the
# substring's letters.
def first_failing(s, words, min_len=3):
    # smallest counterexample: lengths from short to long
    sigs = {signature(w) for w in words}
    lengths = {len(w) for w in words}
    for length in range(min_len, len(s) + 1):
        # no word has this length: the first window fails
        if length not in lengths:
            return s[:length]
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if signature(piece) not in sigs:
                return piece
    return None
```

```
if __name__ == "__main__":
    words = {"cat", "act", "tac", "tact", "cats"}
    assert all_anagram_words("cat", words)
    assert all_anagram_words("tca", words)
    assert not all_anagram_words("tcaz", words)
    assert all_anagram_words_any_alphabet("cat", words)
    assert first_failing("cat", words) is None
    # "catz" fails at length 3? "atz" is not an anagram
    assert first_failing("catz", words) == "atz"
    assert all_anagram_words("ab", set())
    assert first_failing("aaaa", {"aaa"}) == "aaaa"
```

FUNCTION AND TEST CASES

Signature

```
def solution(s: str, words: list[str]) -> bool:
```

Sample call

```
solution("act", ["cat", "dog"])
```

Test cases

Input	Expected
("act", ["cat", "dog"])	True
("abca", ["cab", "abca", "zzz"])	True

STUDY PATH: question 155 of 167 (more practice)

Priority score 2.5 | Sorting and divide and conquer | asked 119 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 3 of 4

Streaming Points: Emit K Within Distance

ASKED 2026-06-23 | 119 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 199/228

ABOUT THIS QUESTION

Topic: More practice - Sorting and divide and conquer

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-points-k-within-distance

QUESTION

Process a stream of float positions. Whenever three points satisfy the distance condition, output and remove those three points; a follow-up generalizes from 3 points to K points. The important ambiguity is whether the condition means a span of length **d** or pairwise distance within **d**.SWEdata-structuresorting sliding-windowmediumambiguous-specFrequencySingle reportLast asked2026-06-23Stageonsite-coding

APPROACH

Practice range queries over sorted points with insertion and deletion.
Write the K-generalization in words before coding: identify the local candidate window, test the predicate, then remove exactly the emitted points.
Rehearse explaining float comparison, tie-breaking, and what to do when multiple groups qualify.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the condition means all K points fit in a span of length d or every pair is within d (on a line these are equivalent), whether to emit as soon as a group exists, and how to choose among overlapping groups.

BRUTE FORCE TO SAY OUT LOUD

Keep a list, and after each point sort it and scan for a group. O(n log n) per point.

OPTIMAL IDEA

Keep points in a sorted list (bisect.insort). After inserting x only windows containing x can be new, so look at the K-wide windows of sorted neighbors around its index; if the largest minus the smallest in a window is at most d, emit and remove those K points. O(log n) to locate plus the window size.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

Exactly d apart, duplicates, points arriving out of order, two overlapping candidate groups, K = 1.

LIKELY FOLLOW-UPS

General K (the same code), 2D points, or a time-based expiration.

PITFALLS

Using epsilon-free float comparison blindly. Checking only the K points directly before x.

HOW TO THINK ABOUT IT

On a line, K points are all within distance d exactly when the largest minus the smallest is at most d, so only K consecutive points in sorted order can form a group. Keep the points sorted and look only where something changed: a window that contains the new point, or the seam left by removed points.

Basis: Written. Assumed rules (confirm them): after each new point, while some K consecutive points (in sorted order) span at most d, emit the lowest such group and remove it; on a line 'all within d' and 'span at most d' are the same condition.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
points = []
def on_point(x):
    points.append(x)
    points.sort()
    emitted = []
    while True:
        group = find_group()
        if group is None:
            return emitted
        emitted.append(group)
        remove(points, group)
def find_group():
    for i in range(len(points) - k + 1):
        if points[i + k - 1] - points[i] <= d:
            return points[i:i + k]
    return None
```

TIME	SPACE
O(n log n) per point	O(n)

WHY IT IS SLOW

Every arrival re-sorts and rescans the whole list, O(n log n). A sorted list with binary search checks only the windows around the new point.

OPTIMAL SOLUTION · Sorted list, check only windows touching the change

TIME	SPACE
O(log n + K) per check, plus list shifting	O(n)

OPTIMAL SOLUTION · Sorted list, check only windows touching the change

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if 'within d' means every pair, and points are in 2D?

On a line pairwise distance equals the span, so nothing changes. In 2D it becomes a clique problem: candidates must be neighbours in a spatial grid of cell size d, and you test the K-subset of nearby points; that is much harder, so ask whether the 1D reading is intended.

FOLLOW-UP 2 Floating-point coordinates.

Compare with the exact subtraction hi - lo <= d (not by building a range around the target), and agree on an epsilon policy only if the interviewer wants one. Do not compare floats for equality.

FOLLOW-UP 3 Several groups qualify at once.

Agree on a rule (lowest start, as coded) so output is deterministic, and note that emitting one group can create or destroy others.

FOLLOW-UP 4 The condition is 'every pair within d', not 'span at most d'.

On a line they are the same: the largest pairwise distance is max - min, so checking the span is enough (verified against checking all pairs). In 2D they differ and become a clique problem.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Streaming Points: Emit K Within Distance
#
# Process a stream of float positions. Whenever three
# points satisfy the distance condition, output and remove
# those three points; a follow-up generalizes from 3
# points to K points. The important ambiguity is whether
# the condition means a span of length d or pairwise
# distance within d.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if 'within d' means every pair, and points are
# in 2D?
# 2. Floating-point coordinates.
# 3. Several groups qualify at once.
# 4. The condition is 'every pair within d', not 'span at
# most d'.
#
# INTERVIEW NOTES
# On a line, K points are all within distance d exactly
# when the largest minus the smallest is at most d, so
# only K consecutive points in sorted order can form a
# group. Keep the points sorted and look only where
# something changed: a window that contains the new point,
# or the seam left by removed points.
#
# BRUTE FORCE (say it first): Re-sort and scan every
# window after each point
# Sort all points, slide a window of K, and emit the
# lowest group that fits, repeating until none does.
# Obviously correct; the whole list is re-examined for
# every new point. Time O(n log n) per point. Space O(n).


from bisect import bisect_left, insort


# BRUTE FORCE (say it first): Re-sort and scan every
# window after each point | time O(n log n) per point
class PointStreamBrute:
    def __init__(self, k, d):
        self.k, self.d, self.pts = k, d, []

    def add(self, x):
        self.pts.append(x)
        out = []
        while True:
            self.pts.sort()
            for i in range(len(self.pts) - self.k + 1):
                if (
                    self.pts[i + self.k - 1]
                    - self.pts[i]
                    <= self.d
                ):
                    out.append(self.pts[i : i + self.k])
                    del self.pts[i : i + self.k]
                    break
            else:
                return out


# BASE QUESTION (optimal): Sorted list, check only windows
# touching the change
# Idea: Insert with bisect. Because no window existed
# before, a new window must contain the new point, so test
# the K windows that include its position.
class PointStream:
    def __init__(self, k, d):
        # emit when k points fit in a span of d
        self.k, self.d = k, d
        # all points still waiting, kept sorted
        self.points = []

    def window_start(self, pos):
        # first window of k neighbours that includes pos
        lo = max(0, pos - self.k + 1)
        hi = min(pos, len(self.points) - self.k)
        for i in range(lo, hi + 1):
            last = self.points[i + self.k - 1]
            span = last - self.points[i]
            if span <= self.d:
                return i
        return None

    def add(self, x):
        # insert in order, then emit every window made by
        # x
        insort(self.points, x)
        out, pos = [], bisect_left(self.points, x)
        while True:
            i = self.window_start(pos)
```

```
            if i is None:
                return out
            # remove the group so it is emitted only once
            out.append(self.points[i:i + self.k])
            del self.points[i:i + self.k]
            pos = i


# FOLLOW-UP 4: The condition is 'every pair within d', not
# 'span at most d'.
# Answer: On a line they are the same: the largest
# pairwise distance is max - min, so checking the span is
# enough (verified against checking all pairs). In 2D they
# differ and become a clique problem.
def all_pairs_within(group, d):
    # "every pair within d": on a line this equals the
    # span
    pairs = [(a, b) for a in group for b in group]
    return all(abs(a - b) <= d for a, b in pairs)


if __name__ == "__main__":
    s = PointStream(3, 1.0)
    assert s.add(0.5) == []
    assert s.add(5.0) == []
    assert s.add(1.0) == []
    # 0.5, 1.0, 1.4 span 0.9 <= 1.0
    out = s.add(1.4)
    assert out == [[0.5, 1.0, 1.4]]
    # the far point is still waiting
    assert s.points == [5.0]
    # on a line, span and pairwise checks agree
    for group in out:
        assert all_pairs_within(group, 1.0)
        assert max(group) - min(group) <= 1.0
    # k = 2 groups closest neighbours as they appear
    t = PointStream(2, 0.5)
    assert t.add(1.0) == []
    assert t.add(1.4) == [[1.0, 1.4]]
```

NOTES FROM CANDIDATES

A balanced ordered set / TreeSet-style structure is a natural fit because each new point only needs nearby neighbors.
Be careful with floating-point comparison and epsilon policy.
A wrong clarification can lead to checking subset(target, target + d) when the interviewer expects a wider candidate range around the target. Confirm the predicate with a tiny numeric example before coding.

PREPARATION

Practice range queries over sorted points with insertion and deletion.
Write the K-generalization in words before coding: identify the local candidate window, test the predicate, then remove exactly the emitted points.
Rehearse explaining float comparison, tie-breaking, and what to do when multiple groups qualify.

STUDY PATH: question 156 of 167 (more practice)

Priority score 3.11 | Sorting and divide and conquer | asked 332 days ago

HARD | PracHub | Technical Screen | 4 of 4

★ TOP 75: rank 10 in the Medium and Hard priority list

★ Find safe travel intervals between planet influences

POSTED 2025-11-22 | 332 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 65.2 | 30 DAYS 41.1 | PRIORITY 9/228

ABOUT THIS QUESTION

Topic: More practice - Sorting and divide and conquer

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #56 Merge Intervals

LeetCode match: LC 56 Merge Intervals (then take gaps)

Posted on PracHub: 2025-11-22 (updated 2026-05-23)

Original: https://prachub.com/coding-questions/find-safe-travel-intervals-between-planet-influences

QUESTION

You are planning a space route along a one-dimensional line (the x-axis). You are given a list of planets. Each planet is represented by an integer pair ((c, r)):

- **c** is the coordinate of the planet's center on the x-axis.
- **r** is a non-negative integer radius of its gravitational influence. A point **x** on the x-axis is **unsafe** for a space traveler if it lies within the gravitational influence of at least one planet: [|x - c_i| \le r_i \quad \text{for some planet } i] Equivalently, each planet makes the interval ([c_i - r_i,\; c_i + r_i]) unsafe. Task:
- Consider the entire real line as the possible travel path.
- Compute all **maximal continuous intervals** on this line where the traveler is **safe**, i.e., points not covered by any planet's unsafe interval.
- Return these safe intervals as a sorted list of disjoint intervals. Representation details:
- Represent each safe interval as a pair **`(start, end)`** with **`start < end`**.
- Intervals may be unbounded on the left or right:
- Use **`-inf`** for an interval unbounded to the left (e.g., **`(-inf, a)`**).
- Use **`+inf`** for an interval unbounded to the right (e.g., **`(b, +inf)`**). You may assume the input list of planets can be in any order and may contain overlapping or nested gravitational ranges. Return the safe intervals in increasing order of **`start`**.

QUESTION

You are planning a route along the real x-axis. Each planet is given as a pair (c, r), where c is its center and r is a non-negative radius of gravitational influence. A point x is unsafe if it lies inside at least one planet's influence, meaning |x - c| <= r. So each planet creates an unsafe closed interval [c - r, c + r]. Your task is to return all maximal continuous safe intervals on the real line: regions not covered by any unsafe interval. Represent each safe interval as a tuple (start, end), where the interval means start < x < end. Because JSON/Python literals do not have a direct infinity literal, use None to represent an unbounded side:

- (None, a) means (-infinity, a)
- (b, None) means (b, +infinity)
- (None, None) means the entire real line Return the safe intervals in increasing order. Overlapping, nested, and touching unsafe intervals should be treated as one merged unsafe region.

Constraints

0 <= len(planets) <= 200000-10^9 <= c <= 10^90 <= r <= 10^9

EXAMPLES

```
Input: []
Expected Output: [(None, None)]
```

Explanation: With no planets, no point is unsafe, so the entire real line is safe.

```
Input: [(3, 1), (10, 2)]
Expected Output: [(None, 2), (4, 8), (12, None)]
Explanation: The unsafe intervals are [2, 4] and [8,
    12]. Their complement is (-infinity, 2), (4, 8),
    and (12, +infinity).
```

Hints

First convert every planet (c, r) into the unsafe interval [c - r, c + r].Sort the unsafe intervals and merge all overlapping or touching ones. The safe intervals are the gaps before, between, and after the merged intervals.

APPROACH

Turn each planet into [c-r, c+r], sort, merge overlaps, and the safe intervals are the open gaps between merged intervals plus the two infinite ends. Watch endpoint inclusivity (unsafe intervals are closed).

HOW TO THINK ABOUT IT

The safe points are the complement of the union of the unsafe intervals. Merge the unsafe intervals first (sort by start, extend while they overlap or touch), and the safe intervals are exactly the gaps between consecutive merged intervals plus the two unbounded ends. Endpoints themselves are unsafe because the influence is closed (<=).

Basis: Interval complement (LeetCode 56, Merge Intervals, then take the gaps). Each planet (c, r) makes the closed interval [c - r, c + r] unsafe; safe intervals are the open gaps, possibly unbounded.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(planets):
    pts = set()
    for c, r in planets:
        pts |= {c - r, c + r}
    bounds = [-INF] + sorted(pts) + [INF]
    safe = []
    for a, b in zip(bounds, bounds[1:]):
        mid = midpoint(a, b)
        if not any(abs(mid - c) < r
                   for c, r in planets):
            safe.append((a, b))
    return safe
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each midpoint is tested against every planet, O(n^2). Merging the unsafe intervals after sorting and reading the gaps is O(n log n).

OPTIMAL SOLUTION · Merge the unsafe intervals, then read off the gaps

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the part between two given positions matters (a route from A to B).

Intersect each safe interval with [A, B] and keep the non-empty ones. Or clip the planets first; the merge-and-gap logic is the same.

FOLLOW-UP 2 Are the endpoints safe?

The statement uses <=, so the endpoints of an unsafe interval are unsafe and the safe intervals are open. If the influence were strict (<), touching intervals would leave a single safe point between them and the merge test would use < instead of <=.

FOLLOW-UP 3 Planets arrive over time.

Keep the merged unsafe intervals in a sorted structure; an insert merges neighbours (like the stream-of-intervals problem) and the safe intervals are always the gaps.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find safe travel intervals between planet
# influences
#
# You are planning a space route along a one-dimensional
# line (the x-axis). You are given a list of planets. Each
# planet is represented by an integer pair \((c, r)\): - c
# is the coordinate of the planet's center on the x-axis.
# - r is a non-negative integer radius of its
# gravitational influence. A point x on the x-axis is
# **unsafe** for a space traveler if it lies within the
# gravitational influence of at least one planet: \[ |x -
# c_i| \le r_i \quad \text{for some planet } i \]
# Equivalently, each planet makes the interval \([c_i -
# r_i,\; c_i + r_i]\) unsafe. Task: - Consider the entire
# real line as the possible travel path. - Compute all
# **maximal continuous intervals** on this line where the
# traveler is **safe**, i.e., points not covered by any
# planet's unsafe interval. - Return these safe intervals
# as a sorted list of disjoint intervals.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the part between two given positions matters (a
# route from A to B).
# 2. Are the endpoints safe?
# 3. Planets arrive over time.
#
# INTERVIEW NOTES
# The safe points are the complement of the union of the
# unsafe intervals. Merge the unsafe intervals first (sort
# by start, extend while they overlap or touch), and the
# safe intervals are exactly the gaps between consecutive
# merged intervals plus the two unbounded ends. Endpoints
# themselves are unsafe because the influence is closed
# (<=).
#
# BRUTE FORCE (say it first): Test the midpoint of every
# segment between critical points
# Collect all interval endpoints, and for every
# consecutive pair (and the two outer rays) test whether
# the midpoint is covered by some planet. Adjacent safe
# segments are separate because their shared endpoint is
# unsafe. Correct and the oracle; every midpoint is tested
# against every planet. Time O(n^2). Space O(n).
```

```
# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first): Test the midpoint of every
# segment between critical points | time O(n^2)
def safe_intervals_brute(planets):
    iv = [(c - r, c + r) for c, r in planets]
    pts = sorted({x for a, b in iv for x in (a, b)})
    if not pts:
        return [(-INF, INF)]

    def safe(x):
        return not any(a <= x <= b for a, b in iv)

    out = []
    if safe(pts[0] - 1):
        out.append((-INF, pts[0]))
    for p, q in zip(pts, pts[1:]):
        if safe((p + q) / 2):
            out.append((p, q))
```

```
    if safe(pts[-1] + 1):
        out.append((pts[-1], INF))
    return out


# HELPERS for the base question
def unsafe_blocks(planets):
    # planet (c, r) makes [c - r, c + r] unsafe; merge
    # overlaps
    merged = []
    spans = sorted((c - r, c + r) for c, r in planets)
    for a, b in spans:
        # sharing even one point joins two blocks
        if merged and a <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], b)
        else:
            merged.append([a, b])
    return merged


# HELPERS for follow-up 3
def safe_intervals_from(blocks):
    if not blocks:
        return [(-INF, INF)]
```

```
    out = [(-INF, blocks[0][0])]
    for prev, nxt in zip(blocks, blocks[1:]):
        out.append((prev[1], nxt[0]))
    out.append((blocks[-1][1], INF))
    return out


# BASE QUESTION (optimal): Merge the unsafe intervals,
# then read off the gaps
# Idea: Sort the unsafe intervals by start and merge any
# that overlap or touch (they share a point, which is
# unsafe).
def safe_intervals(planets):
    # the gaps between unsafe blocks (open intervals)
    merged = unsafe_blocks(planets)
    if not merged:
        return [(-INF, INF)]
    out = [(-INF, merged[0][0])]
    for prev, nxt in zip(merged, merged[1:]):
        out.append((prev[1], nxt[0]))
    out.append((merged[-1][1], INF))
    return out


# FOLLOW-UP 1: Only the part between two given positions
# matters (a route from A to B).
# Answer: Intersect each safe interval with [A, B] and
# keep the non-empty ones. Or clip the planets first; the
# merge-and-gap logic is the same.
def safe_between(planets, a, b):
    # a route from a to b: clip every safe interval to [a,
    # b]
    out = []
    for lo, hi in safe_intervals(planets):
        lo, hi = max(lo, a), min(hi, b)
        if lo < hi:
            out.append((lo, hi))
    return out


# FOLLOW-UP 3: Planets arrive over time.
# Answer: Keep the merged unsafe intervals in a sorted
# structure; an insert merges neighbours (like the stream-
# of-intervals problem) and the safe intervals are always
# the gaps.
```

```
class LiveRoute:
    """Planets arrive over time: keep blocks merged."""
    def __init__(self):
        self.blocks = []

    def add(self, c, r):
        a, b = c - r, c + r
        keep = []
        for s, e in self.blocks:
            if e < a or s > b:
                keep.append([s, e])
            else:
                # overlaps the new block: absorb it
                a, b = min(a, s), max(b, e)
        keep.append([a, b])
        self.blocks = sorted(keep)

    def safe(self):
        return safe_intervals_from(self.blocks)


if __name__ == "__main__":
    planets = [(0, 2), (10, 1), (3, 1)]
    # unsafe: [-2, 4] and [9, 11]
    assert safe_intervals(planets) == [(-INF, -2), (4, 9),
                                       (11, INF)]
    assert safe_intervals([]) == [(-INF, INF)]
    # touching blocks merge: [0, 2] and [2, 4]
    assert safe_intervals([(1, 1), (3, 1)]) == [
        (-INF, 0), (4, INF)]
    # only the part of the route between 5 and 20 matters
    route = safe_between(planets, 5, 20)
    assert route == [(5, 9), (11, 20)]
    live = LiveRoute()
    for c, r in planets:
        live.add(c, r)
    assert live.safe() == safe_intervals(planets)
    # a big planet at 6 swallows both blocks
    live.add(6, 3)
    assert live.safe() == [(-INF, -2), (11, INF)]
```

FUNCTION AND TEST CASES

Signature

```
def solution(planets: list[list[int]]) -> list[list[int]]:
SAMPLE INPUT
[]
```

Test cases

Input	Expected
`[]`	`[(None, None)]`
`[(3, 1), (10, 2)]`	`[(None, 2), (4, 8), (12, None)]`